

Московский государственный технический университет
Факультет ИУ «Информатика и системы управления»
Кафедра ИУ-7 «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе №1

«Венгерский метода решения задачи о назначениях»

по дисциплине

«Методы вычислений»

Вариант 9

Выполнили: Салженикин Д.А.
Группа: ИУ1-11М

Проверил: Власов П.А.

Работа выполнена: 29/10/2025
Отчет сдан: 05/10/2025
Оценка:

Общий порядок выполнения лабораторной работы

1. Содержательная и математическая постановки задачи о назначениях.
2. Краткое описание венгерского метода.
3. Текст программы.
4. Результаты расчетов для задачи из индивидуального варианта.

1. Содержательная и математическая постановки задачи о назначениях

В распоряжении работодателя имеется n работников и n видов работ. Стоимость выполнения i-ой работы j-ым работником составляет $c_{ij} \geq 0$ единиц.

Требуется распределить работы между работниками так, чтобы каждый работник выполнял ровно один вид работ, а общая стоимость выполнения всех видов работ была минимальной.

Введём управляемые переменные:

$$x_{ij} = \begin{cases} 1, \text{если } i-\text{ую работу выполняет } j-\text{ый сотрудник} \\ 0, \text{иначе} \end{cases} \quad (1)$$

Тогда, математическая постановка задачи о назначениях:

$$\begin{cases} f = \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij} \to min\,, \\ \sum_{j=1}^{n} x_{ij} = 1, \qquad i = \overline{1,n}, \\ \sum_{i=1}^{n} x_{ij} = 1, \qquad j = \overline{1,n}, \\ x_{ij} \in \{0,1\}, i,j = \overline{1,n}, \end{cases} \quad (2)$$

где f – целевая функция;

x_{ij} – элементы матрицы назначений X.

c_{ij} – элементы матрицы стоимостей C.

Общую стоимость выполнения всех работ задает первое выражение. Второе выражение показывает, что i-ую работу выполняет ровно один исполнитель. А третье выражение показывает, что j-ый исполнитель выполняет ровно одну работу.

Исходные данные матрицы стоимостей для 9 варианта:

$$C = \begin{bmatrix} 4 & 7 & 1 & 5 & 5 \\ 6 & 8 & 3 & 7 & 6 \\ 6 & 4 & 5 & 7 & 7 \\ 4 & 2 & 3 & 4 & 9 \\ 8 & 1 & 8 & 3 & 8 \end{bmatrix} \quad (3)$$

2. Краткое описание венгерского метода

Введём определение системы независимых нулей:

Системой независимых нулей (СНН) будем называть такой набор нулей матрицы C, никакие два элемента которого не стоят ни в общей строке, ни в общем столбце.

Венгерский метод:

1. Из каждого столбца матрицы C вычтем его наименьший элемент.
2. Из каждой строки матрицы C вычтем её наименьший элемент.
3. Строим начальную систему независимых нулей (СНН): просматриваем матрицу стоимостей по столбцам слева направо. Если в столбце встречается нуль, в одной строке с которым нет 0^*, то отмечаем его 0^* и переходим к следующему столбцу. k – количество нулей в построенной СНН.
4. Если k < n:
   - 4.1. Отмечаем “+” столбцы матрицы стоимостей, в которых есть 0^*.
   - 4.2. Если среди невыделенных элементов есть 0:
     - 4.2.1. Отмечаем $0'$
     - 4.2.2. Если в одной строке с ним есть 0^*:
       - 4.2.2.1. Снимаем выделение со столбца с этим 0^*, выделяем “+” строку с этим $0'$.
       - 4.2.2.2. Переход к п. 4.2.
     - 4.2.3. Иначе:
       - 4.2.3.1. От текущего $0'$ строим не продолжаемую L-цепочку:

$$0' \xrightarrow{\text{по столбцу}} 0^* \xrightarrow{\text{по строке}} 0' \xrightarrow{\text{по столбцу}} 0^* \to \cdots \to 0'.$$

   - 4.3. Иначе:
     - 4.3.1. Находим наименьший элемент $h > 0$ среди невыделенных.
     - 4.3.2. Вычитаем h из невыделенных столбцов.
     - 4.3.3. Добавляем h к выделенным строкам.
     - 4.3.4. Переход к п. 4.2.
5. Записываем оптимальное решение: в матрицу x_{opt} записываем 1 в те позиции, где в матрице стоимостей стоит 0^*, в остальные позиции пишем 0.
6. $f_{opt} = f(x_{opt})$. $f_{opt} = f(x_{opt})$.
7. Вывод f_{opt}, x_{opt}
8. Конец.

Краткое описание венгерского метода представлено блок-схемой на рисунке 1.

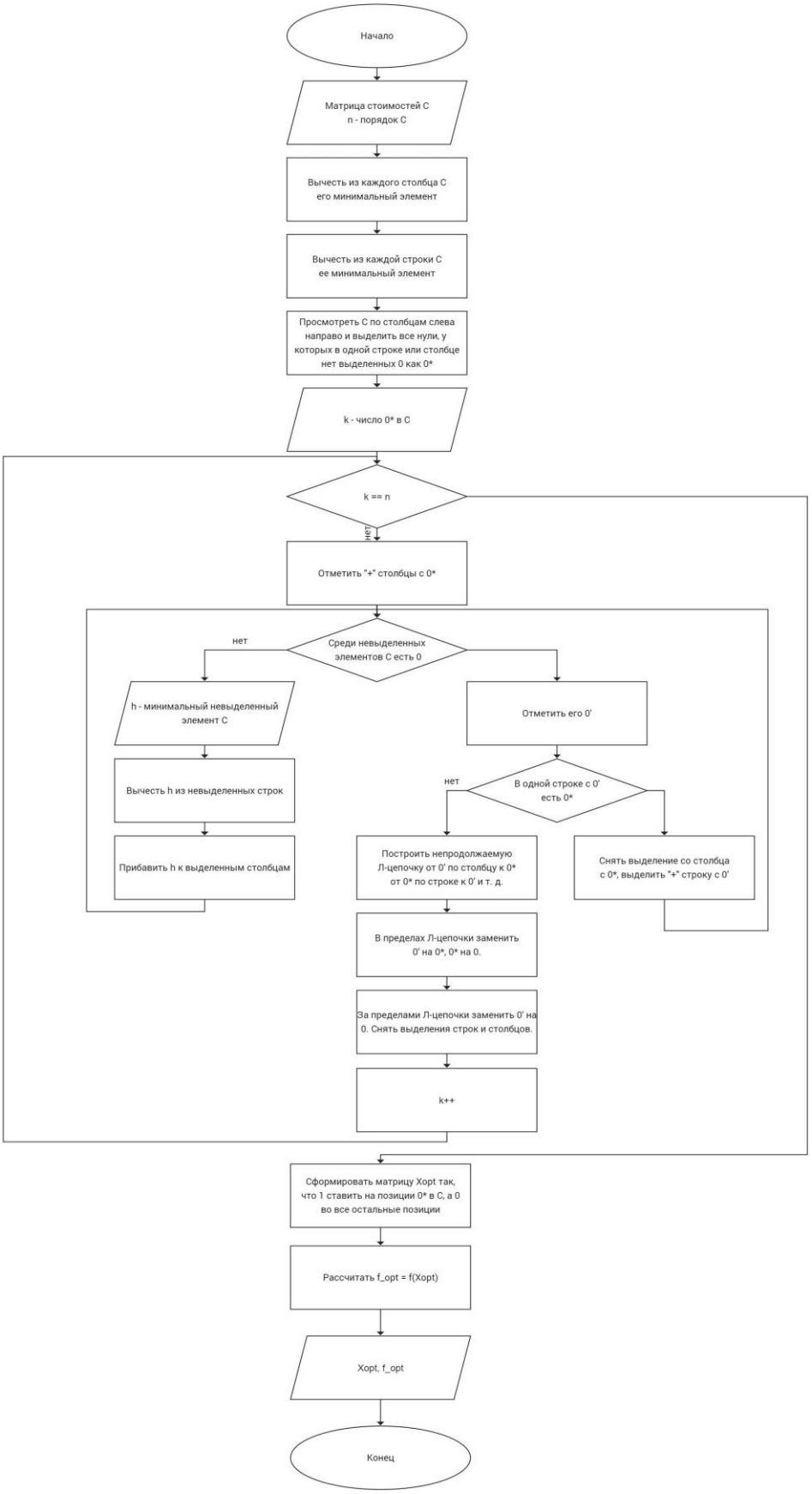


Рис. 1. Блок-схема венгерского метода

3. Текст программы

Текс программы представлен в листинге 1.

Листинг 1 – текст программы

```
clc
clear all

% Исходная матрица стоимостей.
c = [
4 7 1 5 5;
6 8 3 7 6;
6 4 5 7 7;
4 2 3 4 9;
8 1 8 3 8
];

% Матрица для решения задачи максимизации.
c1 = max(c, [], 'all') - c;

marks_min = hungarian(c, true);
marks_max = hungarian(c1, true);

fmin = sum(c.*marks_min, 'all');
fmax = sum(c.*marks_max, 'all');

fprintf("Матрица стоимостей:\n");
print_matrix(c);
fprintf("\nЗадача минимизации.\n");
fprintf("Оптимальное значение функции: %d\nОптимальное решение:\n", fmin);
print_matrix(marks_min);
fprintf("\nЗадача максимизации.\n");
fprintf("Оптимальное значение функции: %d\nОптимальное решение:\n", fmax);
print_matrix(marks_max);

function [marks] = hungarian(source_matrix, debug)
    %Размер исходной матрицы (n x n)
    n = length(source_matrix );

    % Текущий режим (с выводом - true, без вывода - false).
    DEBUG = debug;

    % Отмеченные нули: 1 - "0*", -1 - "0'"
    marks = zeros(n,n);
    % Отмеченные столбцы
    mark_column = zeros(1,n);
    % Отмеченные строки
    mark_row = zeros(1,n);
    % L-цепочка
    l_chain = zeros(2,2);

    % Номер итерации.
    iter = 0;

    % Вычтем из столбцов минимальные элементы.

    source_matrix = source_matrix - min(source_matrix );
    % Вычтем из строк минимальные элементы.
    source_matrix = source_matrix - min(source_matrix , [], 2);
```

```
% Количество элементов в системе независимых нулей.
k = 0;
% Строим начальную систему независимых нулей.
for j = 1:n
    for i = 1:n
        if (source_matrix (i,j) == 0)
            if (max(marks(i, :)) == 0)
                marks(i,j) = 1;
                mark_column (j) = 1;
                k = k+1;
                break;
            end
        end
    end
end

iter = print_siz(DEBUG, iter, source_matrix, marks, l_chain);

while k < n
    has_zeros = false;
    j = 0;
    while ~has_zeros
        j = j+1;
        if j > n
            break;
        end
        if mark_column (j) == 1
            continue
        end
        i = 0;
        while ~has_zeros
            i = i+1;
            if (i > n)
                break;
            end
            if mark_row(i) == 1
                continue
            end

            if source_matrix (i,j) ~= 0 || marks(i,j) == 1
                continue;
            end
            marks(i, j) = -1;
            [mm, jmm] = max(marks(i, :));

            if mm ~= 1
                mark_row = zeros(1, n);
                mark_column = zeros(1, n);
                l_chain = [l_chain; [i j]];
                mark_column (j) = 1;
                jm = j;
                while (true)
                    [m, im] = max(marks(:, jm));
                    if m == 0
                        break;
                    end
                    [m, jmn] = min(marks(im, :));
                    if (m == 0)
                        break;
                    end
                    marks(im, jm) = 0;
                    marks(i, j) = 1;
                    i = im;
```

```
                        j = jmn;
                        l_chain = [l_chain; [im jm]; [im jmn]];
                        jm = jmn;
                    end
                    marks(i,j) = 1;
                    for i = 1:n
                        [m, j] = min(marks(i, :));
                        while m == -1
                            marks(i, j) = 0;
                            [m, j] = min(marks(i, :));
                        end
                    end
                    for j = 1:n
                        if (max(marks(:, j)) == 1)
                            mark_column(j) = 1;
                        end
                    end
                    k = k+1;
                    [iter, l_chain] = print_siz(DEBUG, iter, source_matrix,
marks, l_chain);

                else
                    mark_column (jmm) = 0;
                    mark_row(i) = 1;
                end
                has_zeros = true;
            end
        end
        if has_zeros
            continue;

        end
        source_matrix = make_zeros(source_matrix, mark_row, mark_column);
    end
end

% Преобразуем исходную матрицу так, чтобы в невыделенной части матрицы
появился хотя
% бы один ноль
function [source_matrix] = make_zeros(source_matrix, mark_row, mark_column)
    n = length(source_matrix);
    %Находим минимальный элемент среди невыделенных.
    h = max(max(source_matrix));
    for i = 1:n
        if mark_row(i) == 1
            continue;
        end
        for j = 1:n
            if mark_column(j) == 1
                continue;
            end
            if source_matrix(i, j) < h
                h = source_matrix(i, j);
            end
        end
    end

    % Вычитаем из невыделенных столбцов h.
    for j = 1:n
        if mark_column(j) == 0
            source_matrix(:, j) = source_matrix(:, j) - h;
        end
    end
```

```
    % Прибавляем к выделенным строкам h.
    for i = 1:n
        if mark_row(i) == 1
            source_matrix(i, :) = source_matrix(i, :) + h;
        end
    end
end

% Вывод текущей матрицы с отмеченной системой независимых нулей.
% (system_of_independent_zeros = siz)
function [iter, l_chain] = print_siz(debug, iter, source_matrix, marks,
l_chain)
    if debug
        n = length(source_matrix);
        fprintf("Итерация № %d:\n", iter);
        l_chain = print_l_chain(l_chain);
        fprintf("Текущая матрица:\n");

        for i = 1:n
            for j = 1:n
                fprintf("%d", source_matrix(i,j));
                if marks(i, j) == 1
                    fprintf("*");
                end
                fprintf("\t");
            end
            fprintf("\n");
        end
        fprintf("\n");
    end
    iter = iter + 1;
end

function [] = print_matrix(matrix)
    n = length(matrix);
    for i = 1:n
        for j = 1:n
            fprintf("%d\t", matrix(i,j));
        end
        fprintf("\n");
    end
end

% Вывод L-цепочки.
function [l_chain] = print_l_chain(l_chain)
    n = length(l_chain);

    if n > 2
        fprintf("L-цепочка:\n(%d, %d) ", l_chain(3, 1), l_chain(3, 2));
    if n > 3
        for i = 4:n
            fprintf(" -> (%d, %d) ", l_chain(i, 1), l_chain(i, 2));
        end
    end
    fprintf("\n\n");
    l_chain = zeros(2,2);
    end
end
```

4. Результаты расчетов для задач из индивидуального варианта

Результаты расчетов задачи минимизации представлены на рисунке 2.

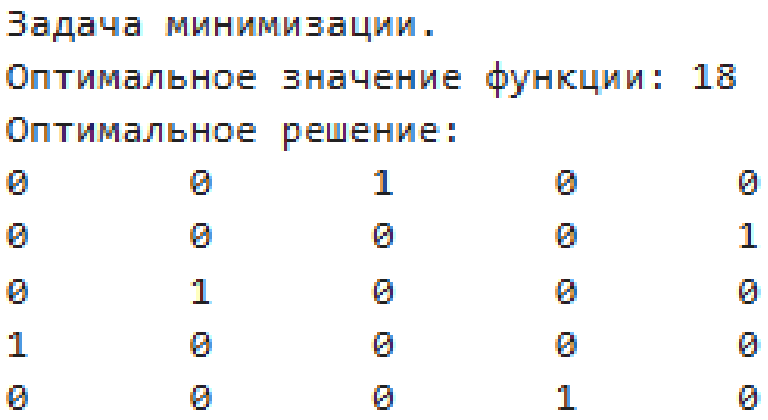


Рис. 2. Результаты расчетов задачи минимизации

Результаты расчетов задачи минимизации представлены на рисунке 3.

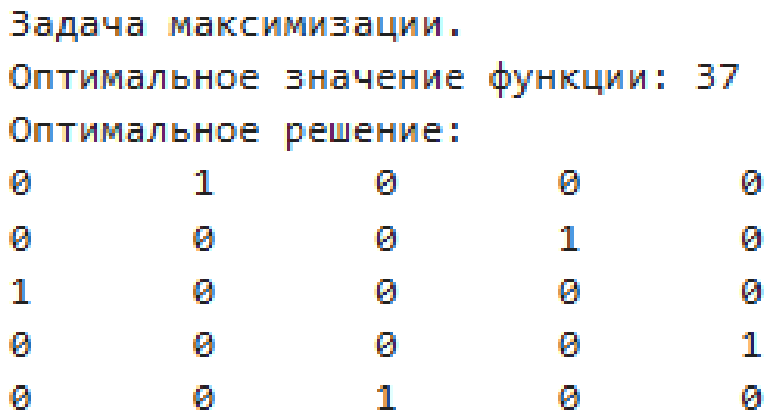


Рис. 3. Результаты расчетов задачи максимизации

Выводы

При выполнении данной работы был изучен венгерский метода решения задачи о назначениях. Также, реализован венгерский метод решения задачи о назначениях в виде программы на ЭВМ. Результаты расчетов продемонстрировали правильное решения для двух решений, как для первого, где задача о назначениях является задачей минимизации, так и для второго, где задача о назначениях является задачей максимизации.